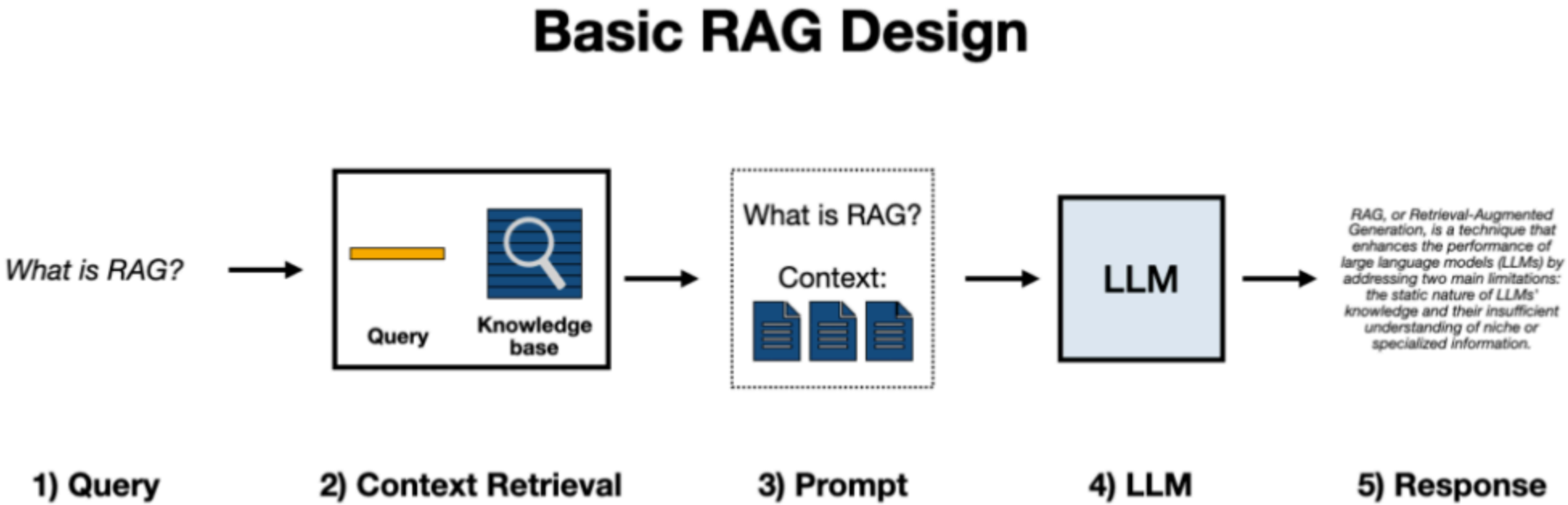
Multimodal RAG

1. 기존 RAG vs Multimodal RAG

기존 Retrieval Augmented Generation

- 검색으로 찾은 근거를 LLM에 함께 전달해 답변 생성
- 처리 과정
  i. 문서를 청크(chunk) 단위로 분할
  ii. 각 청크를 **텍스트 임베딩**으로 변환해 Vector DB에 저장
  iii. 질문이 들어오면 질문도 임베딩 → 유사한 청크 Top-K 검색
  iv. 검색된 청크 + 질문을 LLM에 전달 → 답변 생성





Basic design of a RAG system. Image by author.

기존 RAG의 한계

- 문서를 **텍스트로만 추출**하기 때문에 표·차트·이미지에서 정보가 손실됨
  - 표를 텍스트로 뽑으면 행/열 구분이 무너져 숫자가 뒤섞임
  - 차트는 텍스트 추출 자체가 거의 불가능 (수치가 그림 안에 있음)
  - 스캔된 이미지 문서는 OCR 품질에 따라 정확도가 크게 흔들림
- 즉, "레이아웃 정보"와 "시각 정보"가 파이프라인에서 사라짐

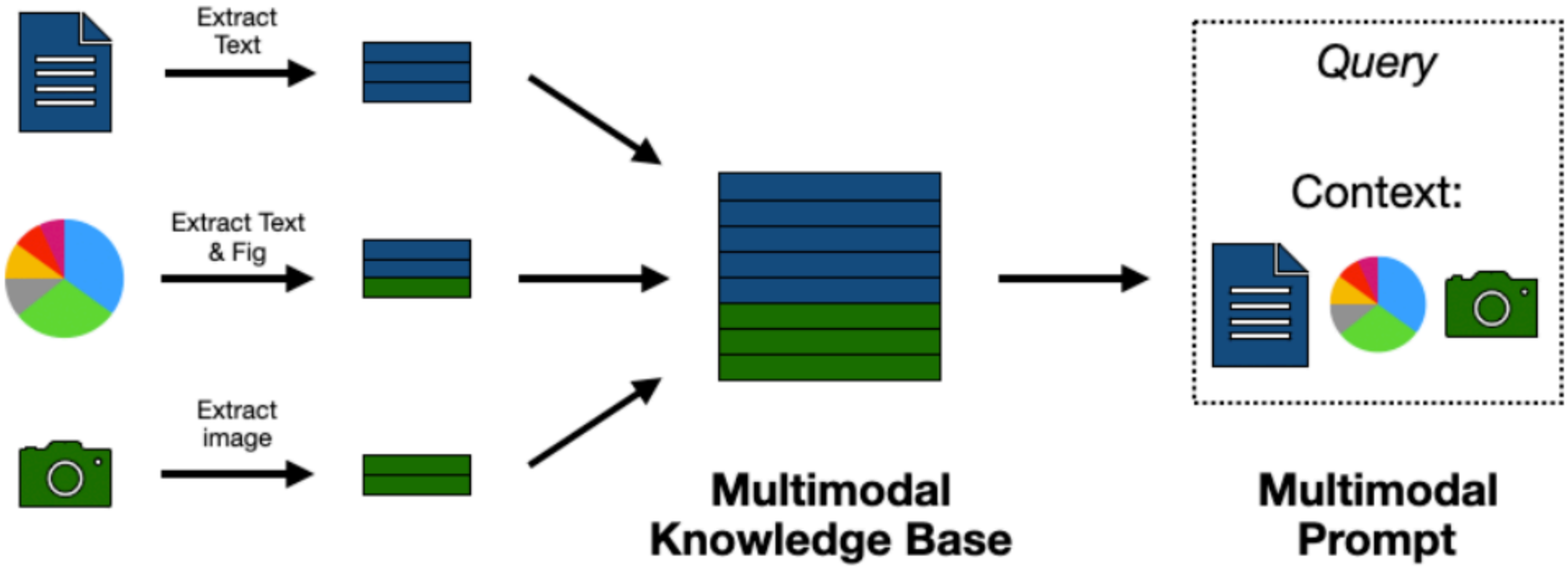
> 보고서/논문/슬라이드처럼 표·그림 비중이 높은 문서일수록 기존 RAG의 정확도가 떨어짐

Multimodal RAG란?

- 기존 텍스트 RAG의 한계를 보완 — **이미지/표/차트까지 임베딩**하여 검색 대상에 포함
  - 텍스트로 옮기며 손실되는 레이아웃·시각 정보를 그대로 보존
- 검색된 이미지/텍스트를 함께 VLM에 전달해 답변 생성



> 일반 RAG가 "텍스트 창고"라면, Multimodal RAG는 "이미지·표·차트까지 포함한 창고"

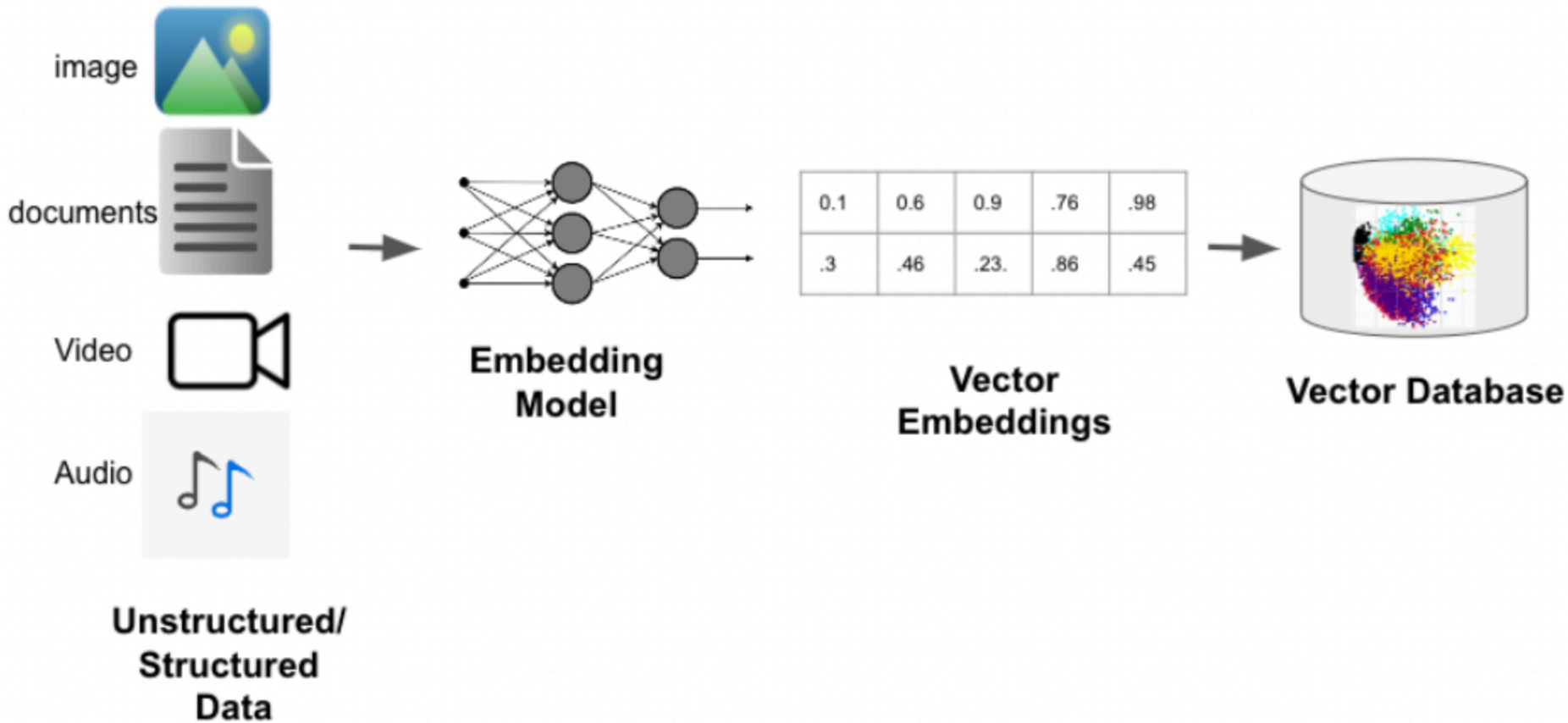


Visual overview of Level 3 of MRAG. Image by author.

기존 RAG vs Multimodal RAG

구분	기존 RAG	Multimodal RAG
검색 대상	텍스트 청크만	텍스트 + 이미지 + 표
표/차트 처리	텍스트 변환 과정에서 정보 손실	이미지 그대로 보존 (또는 요약 후 원본 연결)
임베딩 모델	텍스트 임베딩 모델 1종	텍스트·이미지가 같은 벡터 공간에 있는 멀티모달 임베딩
LLM에 전달하는 것	텍스트만	텍스트 + 이미지
적합한 문서	순수 텍스트 문서	표/차트/이미지가 많은 보고서·논문·슬라이드

2. 이미지 임베딩과 Vector DB


image
documents
Video
Audio
Unstructured/
Structured
Data
Embedding
Model
0.1
0.6
0.9
.76
.98
.3
.46
.23.
.86
.45
Vector
Embeddings
Vector Database

이미지 임베딩 (Image Embedding)

- Multimodal RAG의 핵심 재료 — Vision Encoder 출력을 **고정 길이 벡터**로 변환
- 텍스트 임베딩과 **같은 벡터 공간**에 놓이도록 학습된 모델을 사용해야 텍스트-이미지 교차 검색이 가능
- 대표 모델: CLIP 계열, SigLIP, Gemma 4의 Vision Encoder 출력 활용

```
import torch

image = load_image("product.jpg")
inputs = processor(images=image, return_tensors="pt")

with torch.no_grad():
    vision_outputs = model.get_image_features(**inputs)

embedding = vision_outputs.mean(dim=1)  # (1, hidden_dim) 벡터로 축약
```

Vector DB에 저장

- Vector DB(예: Chroma, FAISS, Milvus)에 이미지 임베딩 + 메타데이터를 함께 저장
- 메타데이터: 원본 파일 경로, 페이지 번호, 캡션, 카테고리 등 → 검색 결과 해석에 필수

```
import chromadb

client = chromadb.Client()
collection = client.create_collection("product_images")

collection.add(
    ids=["img_001"],
    embeddings=[embedding.tolist()],
    metadatas=[{"path": "product.jpg", "category": "shoes"}],
)
```

Similarity Search

- 벡터 간 유사도를 계산해 가장 가까운 K개를 반환 (보통 **Cosine Similarity**)
- 데이터가 많아지면 완전탐색(Brute-force) 대신 **ANN(근사 최근접 이웃)** 인덱스 사용
  - 대표 알고리즘: HNSW, IVF
  - 정확도와 속도를 트레이드오프 — 실무에서는 근사 검색으로 충분한 경우가 많음

```
results = collection.query(
    query_embeddings=[query_embedding.tolist()],
    n_results=5,
)
```

이미지 검색 실전 활용

- 텍스트 질문 → 질문을 임베딩 → 이미지 벡터와 비교(텍스트-이미지 교차 검색)
- 예시: 이커머스 "이 색상, 이 디자인과 비슷한 상품 찾아줘"



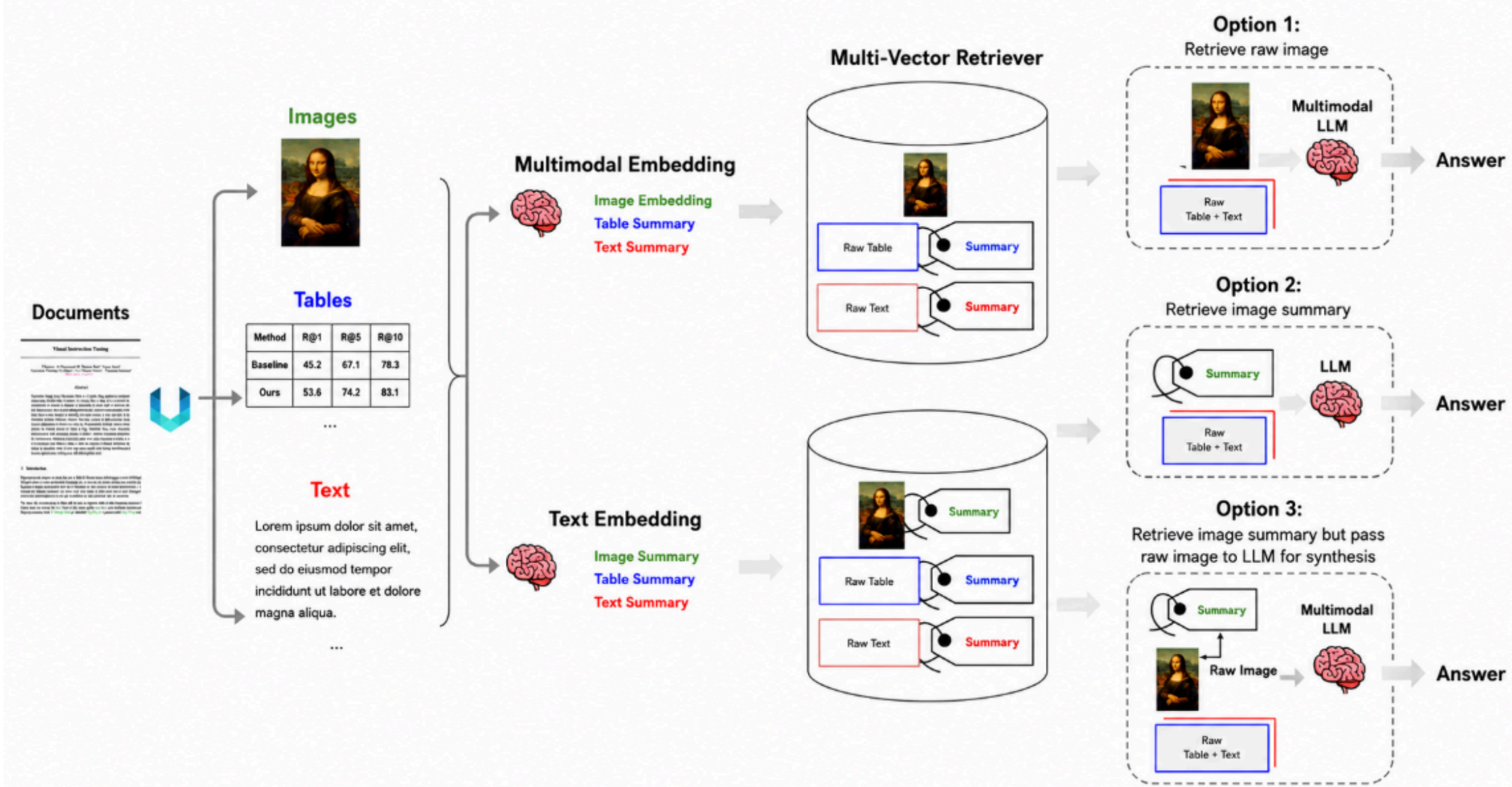
> 검색 정확도는 임베딩 모델의 품질에 크게 좌우됨 — 도메인 특화 파인튜닝 시 정확도 상승

3. Multimodal RAG 파이프라인 구현

Multimodal RAG 구현 3가지 옵션

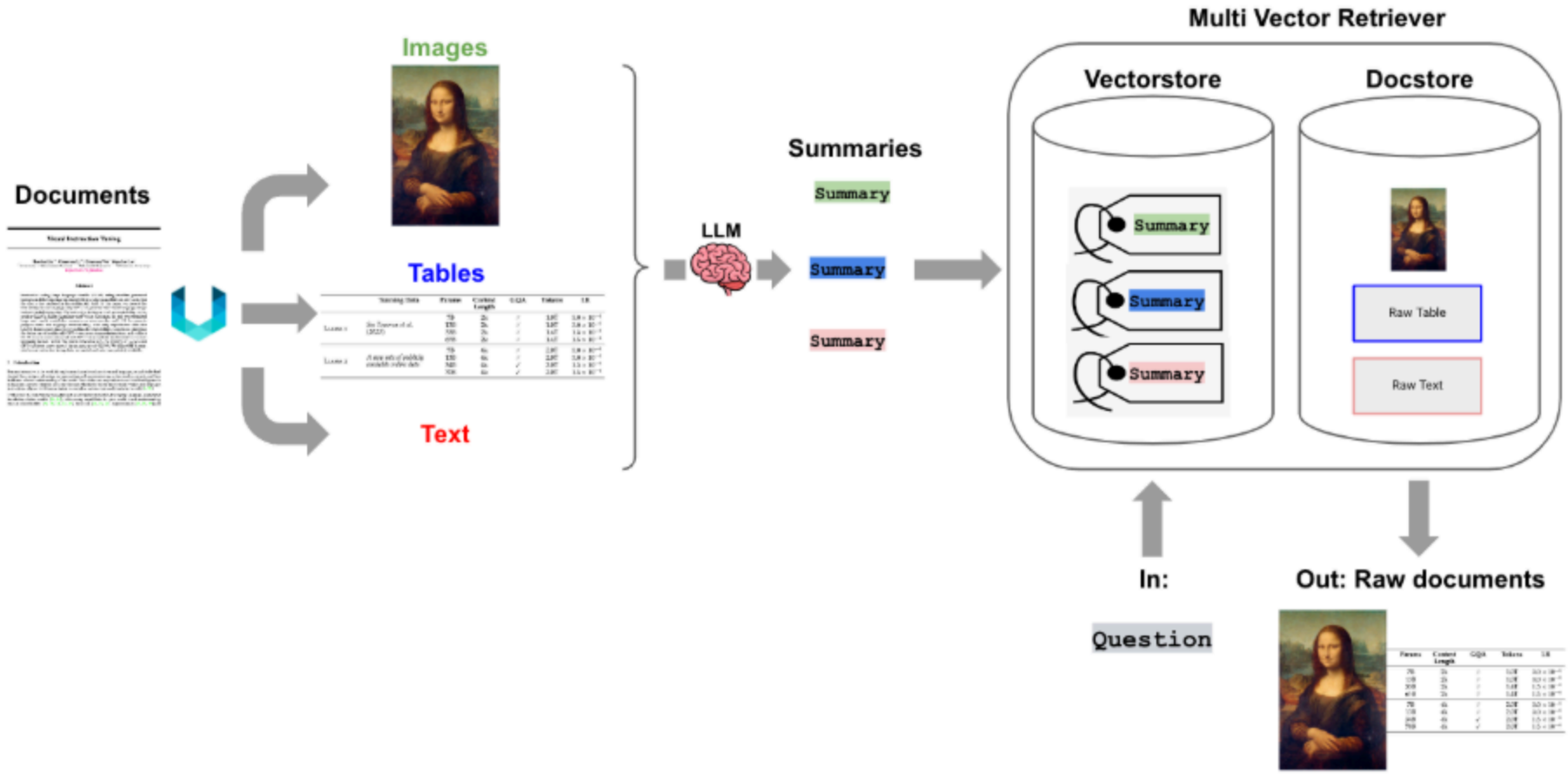
옵션	검색 대상	LLM에 전달하는 내용	특징
① 직접 임베딩	이미지·텍스트 원본 임베딩	원본 그대로	구현 간단, 이미지 임베딩 품질에 검색 정확도 좌우됨
② 요약만 임베딩	이미지 → 텍스트 요약 임베딩	요약 텍스트만	검색은 정확해지지만 **원본 시각 정보가 손실**
③ 요약 임베딩 + 원본 검색	이미지 → 텍스트 요약 임베딩	검색된 요약의 **원본**(이미지)	검색 정확도(텍스트) + 응답 품질(원본) 둘 다 확보

> 이 강의에서는 ③**번(Multi-Vector Retriever 패턴)** 을 기준으로 실습 — 요약은 검색 전용, 원본은 답변 생성 전용으로 역할을 분리


Documents
Images
Tables
Method R@1 R@5 R@10
Baseline 45.2 67.1 78.3
Ours 53.6 74.2 83.1
...
Text
Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua.
...
Multimodal Embedding
Image Embedding
Table Summary
Text Summary
Text Embedding
Image Summary
Table Summary
Text Summary
Multi-Vector Retriever
Raw Table
Summary
Raw Text
Summary
Option 1:
Retrieve raw image
Multimodal LLM
Raw Table + Text
Answer
Option 2:
Retrieve image summary
LLM
Summary
Raw Table + Text
Answer
Option 3:
Retrieve image summary but pass raw image to LLM for synthesis
Summary
Raw Image
Multimodal LLM
Raw Table + Text
Answer

③번(Multi-Vector Retriever 패턴)

- **Vector Store**: 요약 텍스트의 임베딩만 저장 (검색 전용)
- **Document Store**: 원본 이미지/텍스트를 `doc_id` 기준으로 보관 (답변 생성 전용)
- 검색 시 요약으로 매칭 → 매칭된 `doc_id` 로 원본을 꺼내 VLM에 전달


Documents
Images
Tables
Text
LLM
Summaries
Summary
Summary
Summary
Multi Vector Retriever
Vectorstore
Docstore
Summary
Summary
Summary
Raw Table
Raw Text
In:
Question
Out: Raw documents

PDF 검색

- PDF를 텍스트로만 추출하면 **표/그림/레이아웃 정보가 소실**
- 해결책: PDF **각 페이지를 이미지로 렌더링** → 이미지 임베딩으로 색인
- 질문이 들어오면 질문 임베딩과 가장 유사한 페이지(이미지)를 검색 → 해당 페이지 이미지를 VLM에 전달

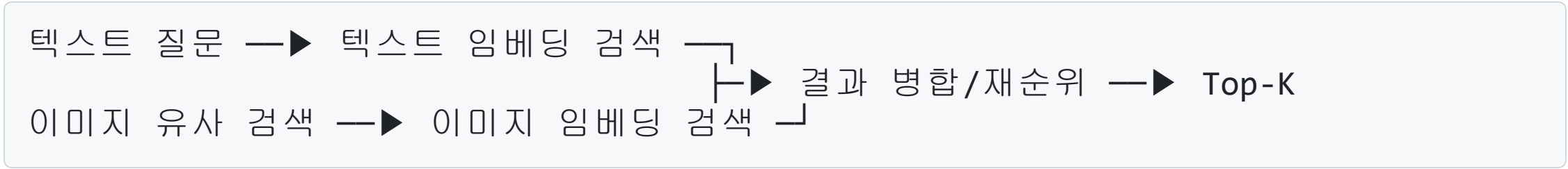
```
from pdf2image import convert_from_path

pages = convert_from_path("report.pdf", dpi=150)
page_embeddings = [embed_image(page) for page in pages]
# page_embeddings를 Vector DB에 저장 → 질문 시 유사도 검색
```

> 텍스트 추출 라이브러리(PyPDF 등)와 병행하면 "텍스트 정확도 + 레이아웃 보존"을 모두 확보

Hybrid Search

- **텍스트 임베딩 검색 + 이미지 임베딩 검색**을 함께 수행해 결과를 결합
- 순수 텍스트 질문(예: "매출 증가율은?")과 순수 시각 질문(예: "빨간색 제품 찾아줘")이 섞인 실무 환경에 대응
- 결합 방식
  - 스코어 합산(Weighted Sum): `score = α·text_score + (1-α)·image_score`
  - 재순위화(Re-ranking): 1차 후보를 모아 VLM으로 최종 순위 재조정



4. 문서 질의응답

PDF QA

- Multimodal RAG로 검색된 **관련 페이지 이미지**를 VLM에 전달해 답변 생성
- 여러 페이지가 검색될 경우, 페이지 순서와 출처(페이지 번호)를 함께 제시하도록 프롬프트 설계

```
아래는 문서에서 검색된 관련 페이지 3장이야.
각 페이지 번호를 인용하며 "연간 매출 목표"가 얼마인지 답해줘.
문서에 없으면 "해당 정보 없음"이라고 답해.
```

이미지 QA

- 단일 이미지에 대한 질의응답 — 검색 없이 바로 VLM에 이미지+질문 전달
- RAG와 결합 시: 유사 이미지들을 함께 컨텍스트로 제공해 **비교 질의**도 가능

```
[검색된 유사 이미지 3장] + 질문:
"이 중에서 재고 이미지와 가장 비슷한 디자인은 몇 번째 이미지야?
 다른 점이 있다면 함께 설명해줘."
```

표 질의응답

- 표는 페이지 이미지 검색만으로는 셀 단위 정확도가 떨어질 수 있음
- 전략: 표를 **마크다운으로 변환한 텍스트본**과 **원본 이미지**를 함께 검색·전달
  - 텍스트본 → 정확한 셀 값 조회
  - 원본 이미지 → 레이아웃/서식 확인 및 텍스트본 검증

```
검색된 표(마크다운 변환본)와 원본 이미지를 함께 참고해서,
"2024년" 열에서 가장 큰 값을 가진 행을 알려줘.
변환 과정에서 누락된 값이 있어 보이면 원본 이미지 기준으로 정정해줘.
```